Pair Trading

~By Raj Karmhe

Introduction to Pair Trading

Introduction to Pair Trading

We have to understand about Pair Trading for that let’s take a scenario and understand it’s working and concepts from it step by step.

Think about a highway which tends to move from a place A to place B and beside that there is a service road running parallelly with it. These two roads tend to show a relation which is they move together and almost parallelly, but think of a scenario where a tree (for some reason it is there) came in the path of service road and hence now the service road will be misaligned, or lose its relation of parallel to highway, just for the time being the tree is in the path. But as the tree got passed now again the highway and service road will follow their usual relationship of being parallel.

Now if we see it in the case of stocks, it can be viewed as some pair of stocks tend to move in a certain type of relationship which they follow most of the time but there are some abnormalities in the relationship due to some event on one particular stock or just some lag of price adjustment of one of two stocks.

Whenever there are such situations we interpret them as trading opportunity, one can trade on this misalignment of prices by seeing which stock has moved more and which one lagged behind and therefore taking short position on the stock which moved more and long position on the stock which lagged behind, but quantitative finance is not about taking on trades just by visually observing abnormalities in prices but making that raw trading opportunity into a well-defined trading signal by multi-level of refinement via statistics/mathematics such as *correlation* and *cointegration* between two stocks, *ADF test, Ornstein Uhlenbeck process, Stochastic differential equation* and many more.

Now let’s see what will be structure through which will be find a trading pair and then trading signals generated by it.

Finding Correlation

The Identification of Trading Pair

Until now we have understood that for two companies to be trading pair they have to be very similar so that they move together but what are those similarities and how do we measure them?

If we think a little about what are things for two companies need to be similar, then we can answer them, they should be: -

- Functioning in same sector, industry.
- Providing similar products or services.
- Serving similar customers
- Having similar role in the country
- Following similar regulatory constraints

Now that we have a sense of in which aspects the companies should be similar, so we can now proceed to thinking how do we say that the two companies out of many such similar companies are good pair and others are not. A good way would be if we can quantize this relationship, but how?

One way is we can use statistical terms such as *covariances* and *standard deviation* on the *daily returns* of stocks of the two companies then use them to find *correlation* to get a sense of does the companies move in the same direction or not. By the use of word direction, I mean to say that does increase in price of one tock leads to increase in other and similarly does it follow the relation for decrease in stock prices.

The Mathematics

The correlation is just ratio of covariance of daily returns of stock prices and product of standard deviation of both the stocks individually. It's value should be greater than 0.80 to be considered good and sometimes we accept values greater than 0.70. It's positive value tells us that they are moving in same direction which is dependent on covariance.

The correlation:

$$\rho_{A,B} = \mathrm{Corr}(R_A, R_B) = \frac{\mathrm{Cov}(R_A, R_B)}{\sigma_{R_A}\,\sigma_{R_B}}$$

The covariance:

$$\mathrm{Cov}(X, Y) = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{n-1}$$

The standard deviation:

$$\sigma_R = \sqrt{\frac{\sum_{i=1}^{n}(R_i - \bar{R})^2}{n-1}}$$

This is the code which I used to find correlation between different stocks:

```
import yfinance as yf

tickers = ['TCS.NS', 'INFY.NS']
data = yf.download(tickers, start="2010-01-01", end="2026-01-
01")['Close'].dropna()

returns = data.pct_change().dropna()

correlation = returns[tickers[0]].corr(returns[tickers[1]])

print(f"The daily return correlation between TCS and Infosys
is: {correlation:.4f}")
```

In particular I used TCS and Infosys as a pair and will be using in future.

The output of this code which is correlation between TCS and Infosys was 0.7107 which is not great but is okay to use.

Calculation of Adaptive Window(Regime Detection)

Now I have founded a pair for trading, I will now have to move on to trading signal generation but for the data sample I had some choices: -

- Using all previous data
- Using certain time period as my lookback window or,
- Making an adaptive window which may vary as per the current market conditions.

So I took the adaptive window path.

We are using adaptive window because when we will be calculating Hedging ratio and CDF(Cumulative Distribution Frequency) values based on KDE(Kernel Density Estimation) curve, we want this quantity to work only on effective data and be free of noises.

An this effective data is double the half life of mean reversion, here half life of mean reversion means the time it would take to revert back to mean.

So the adaptive window is calculated by calculating half life of mean reversion, it can be calculated in following steps:-

1. We first take data (natural log of closing price of stocks) points of past year then we fit it into a linear regression and from where we extract a value beta(Hedge ratio).
2. Then we find the spread of the data using this beta.
3. Then we check whether this spread is behaving like displacement of a spring which reverts back to its mean.
4. Then we fits the spread into a Stochastic Differential Equation which also provide us with speed of mean reversion for spread.
5. Then we calculate half life of mean reversion(ln2/speed of mean reversion) from speed of mean reversion and then get window size(2 times of half life).

Calculating Beta(The Hedge Ratio)

For being market neutral just not mean to divide the equity 50-50 to both the stocks because it assumes if the stock A increases by 1% the stock B also increases by 1% which is not true. Here comes beta it means for every one stock of A we need beta stock of B to be market neutral.

The calculation of beta is exactly same as the calculation in adaptive window was but now not on past year data points but on the adaptive window and nothing different.

ADF Test

Calculation of P value through ADF Test

We have find the window size in which the mean reversion is likely to happen, but how much likely is it? This can be answered in a way by ADF(Augmented Dickey-Fuller) test.

The ADF test check whether spread series is stationary or not, stationary means it is mean reverting and it oscillates around a fixed average.

The ADF test checks for a unit root. Null hypothesis contains a unit root and means that the series is random walk while Alternative hypothesis means the data is stationary and hence is mean reverting.

The ADF test gives us a probability (p value) which tells us how likely is there a presence of unit root. For p-value less than 0.05 we are confident that presence of unit root is less than 0.05 and hence we reject Null Hypothesis.

Calculation of Rarity of a Data point

Till now we have a constraint of p value which tells us when to have a trade, but we have no idea which stock we have to long and which to short. So now we will decide when to long on one short on other, we will establish this by what we first discussed which is whose price is higher and whose is lower but not absolute price value, we will use the spread of natural log prices in our lookback window.

A normal bell curve tells us about rare is a data by distance from its mean which can be found by this equation:

Normal (Gaussian) probability density function (PDF):

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Where:

- μ = mean
- σ = standard deviation

Standardization (z-score):

$$z = \frac{x - \mu}{\sigma}$$

But this curve is too simple to tell whether the data is rare or not. To get more advance we will use Kernel Density Estimation, it makes a bell curve for every data point for a certain lookback window and then sum it all to give the rarity of that data point, it can be calculated by this equation:

Kernel Density Estimation (KDE), 1D:

$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right)$$

Where:

- $x_1, \ldots, x_n$ are the data points in the lookback window
- $K(\cdot)$ is the kernel function

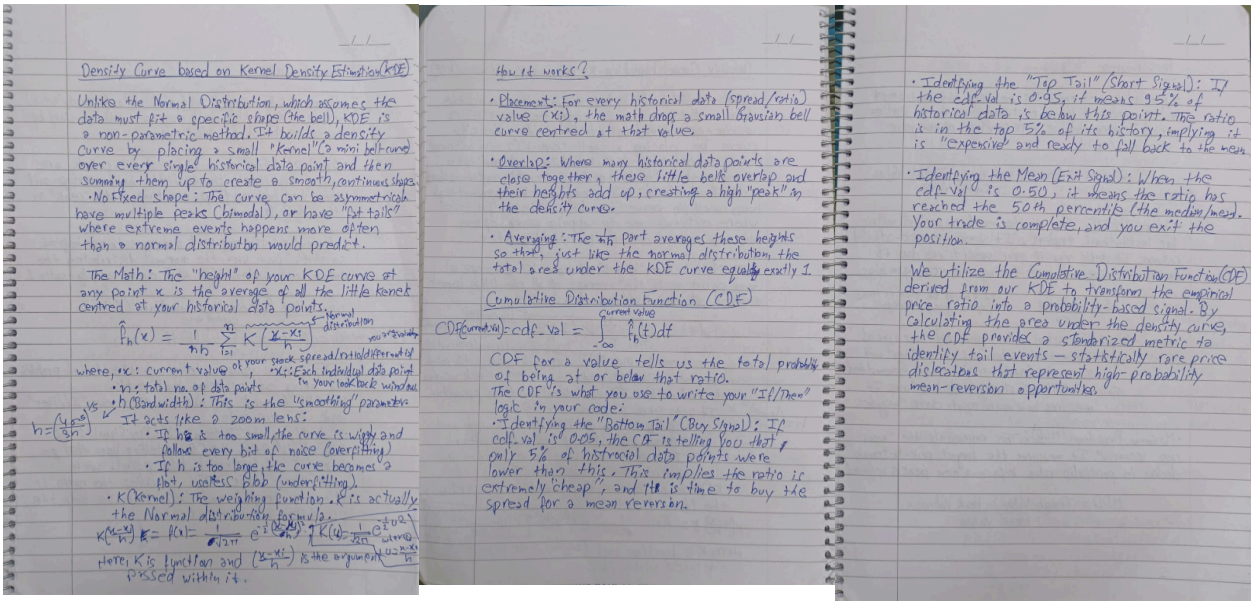
- $h > 0$ is the bandwidth (smoothing parameter)

Common choice (Gaussian kernel):

$$K(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u^2}{2}\right)$$

(Then we plug this $K(u)$ into the KDE formula above.)

I am attaching a part of my notes down here to get a better explanation of what is happening here and from how here we will transcend to generating signals via Cumulative Distribution Frequency(CDF) values.



For stoploss we use 0.01 and 0.99 CDF values to for long and short respectively. And for safety when the stoploss gets triggered and we book loss, so we don't want to book loss again by buying that position again buy satisfying the conditions of entry (stoploss satisfies the condition of entry) so we want to wait until it recovers from that stoploss value so we will add one more condition to entry which will be CDF values greater than 0.025 and lesser than 0.975 for long and short respectively.

Code

```
# ============================================================
==================
# SECTION 1: IMPORTS & LIBRARY SETUP
# ============================================================
==================
# Task: Import necessary libraries for data, math, statistic
s, and plotting.
import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
from scipy.stats import gaussian_kde
import statsmodels.api as sm
from statsmodels.tsa.stattools import adfuller

# ============================================================
==================
# SECTION 2: CONFIGURATION & CONSTANTS
# ============================================================
==================
# Task: Define the universe, timeframes, and strategy constra
ints.
TICKERS = ["TCS.NS", "INFY.NS"]
MARKET_TICKER = "^NSEI"
START_DATE = "2020-01-01"
END_DATE = "2026-02-17"

# Strategy Parameters
MIN_WINDOW = 20
MAX_WINDOW = 200
ESTIMATION_WINDOW = 252
INITIAL_CAPITAL = 100
```

```
# ============================================================
==================
# SECTION 3: DATA INGESTION & PRE-PROCESSING
# ============================================================
==================
# Subsection 3.1: Download & Filter
print(f"Downloading data for {TICKERS}...")
try:
    # auto_adjust=True handles Dividends & Splits automatical
ly
    raw_data = yf.download(TICKERS + [MARKET_TICKER], start=S
TART_DATE, end=END_DATE, auto_adjust=True)
    data = raw_data['Close'].dropna()

    if data.empty: raise ValueError("Empty Data")
except Exception as e:
    print(f"Error: {e}. Please check tickers.")
    exit()

# Subsection 3.2: Prepare Transformations
market_returns = data[MARKET_TICKER].pct_change().dropna()
log_a = np.log(data[TICKERS[0]])
log_b = np.log(data[TICKERS[1]])

# ============================================================
==================
# SECTION 4: STRATEGY INITIALIZATION
# ============================================================
==================
# Task: Create lists and variables to track performance and s
tate.
portfolio = [INITIAL_CAPITAL]
position = 0

# Metric Tracking Lists
trade_returns = []
```

```
trade_durations = []
trade_history = []

# Visualization Lists
dynamic_windows = []
p_value_history = []

# State Variables
entry_equity = 0
entry_idx = 0

print("Running Adaptive Backtest with Full Logic...")

# ============================================================
==================
# SECTION 5: THE BACKTEST LOOP
# ============================================================
==================
# Task: Iterate through time, calculating signals and PnL day
-by-day.
for t in range(ESTIMATION_WINDOW, len(data)):

    # ---------------------------------------------------------
    # Subsection 5.1: Regime Detection (Adaptive Window)
    # ---------------------------------------------------------
    # Task: Determine if the market is fast or slow mean-reve
rting.
    regime_a = log_a.iloc[t-ESTIMATION_WINDOW : t]
    regime_b = log_b.iloc[t-ESTIMATION_WINDOW : t]

    # Calculate Regime Beta
    model_beta = sm.OLS(regime_a, sm.add_constant(regime_b)).
fit()
    beta_regime = model_beta.params.iloc[1]

    # Calculate OU Process (Mean Reversion Speed)
```

```
    spread_regime = regime_a - (beta_regime * regime_b)
    df_ou = pd.DataFrame({'dx': spread_regime.diff(), 'x': sp
read_regime.shift(1)}).dropna()
    model_ou = sm.OLS(df_ou['dx'], sm.add_constant(df_ou
['x'])).fit()
    theta = model_ou.params.iloc[1]

    # Set Window Size based on Half-Life
    if theta < 0:
        half_life = -np.log(2) / theta
        win_size = int(half_life * 2)
    else:
        win_size = MAX_WINDOW

    window_size = max(MIN_WINDOW, min(win_size, MAX_WINDOW))
    dynamic_windows.append(window_size)

    # ---------------------------------------------------------
    # Subsection 5.2: Signal Data Preparation
    # ---------------------------------------------------------
    # Task: Prepare the specific data window for trading sign
als.
    train_a = log_a.iloc[t-window_size : t]
    train_b = log_b.iloc[t-window_size : t]

    # Calculate Trade Beta (Hedge Ratio)
    model_trade = sm.OLS(train_a, sm.add_constant(train_b)).f
it()
    beta_trade = model_trade.params.iloc[1]

    # Construct Spread
    spread_window = train_a - (beta_trade * train_b)
    curr_spread = log_a.iloc[t] - (beta_trade * log_b.iloc
[t])

    # ---------------------------------------------------------
```

```
    # Subsection 5.3: The Gatekeeper (ADF Test)
    # ------------------------------------------------------
    # Task: Filter out trades if the spread is not stationar
y.
    try:
        adf_res = adfuller(spread_window, autolag='AIC')
        p_val = adf_res[1]
    except:
        p_val = 1.0
    p_value_history.append(p_val)

    is_safe = p_val < 0.05

    # ------------------------------------------------------
    # Subsection 5.4: Signal Logic (KDE & CDF)
    # ------------------------------------------------------
    # Task: Generate Buy/Sell signals based on probability.
    prev_pos = position

    if not is_safe:
        position = 0
    else:
        kde = gaussian_kde(spread_window)
        cdf = kde.integrate_box_1d(-np.inf, curr_spread)

        if position == 0:
            if 0.025 < cdf < 0.05:
                position = 1    # Long Entry
            elif 0.95 < cdf < 0.975:
                position = -1   # Short Entry (Fixed Logic)
        elif position == 1:
            if cdf >= 0.50 or cdf < 0.01: position = 0 # Exit
Long
        elif position == -1:
            if cdf <= 0.50 or cdf > 0.99: position = 0 # Exit
Short
```

```
    # ------------------------------------------------------
    # Subsection 5.5: Trade Lifecycle Tracking (Entry)
    # ------------------------------------------------------
    # Task: Capture equity and time when a new trade opens.
    if prev_pos == 0 and position != 0:
        entry_equity = portfolio[-1]
        entry_idx = t

    # ------------------------------------------------------
    # Subsection 5.6: PnL Calculation (Market Neutral)
    # ------------------------------------------------------
    # Task: Calculate daily PnL maintaining Beta Neutrality.
    ret_a = (data[TICKERS[0]].iloc[t] / data[TICKERS[0]].iloc
[t-1]) - 1
    ret_b = (data[TICKERS[1]].iloc[t] / data[TICKERS[1]].iloc
[t-1]) - 1

    if prev_pos == 1:
        pnl = (1.0 * ret_a) - (beta_trade * ret_b)
    elif prev_pos == -1:
        pnl = (-1.0 * ret_a) + (beta_trade * ret_b)
    else:
        pnl = 0

    portfolio.append(portfolio[-1] * (1 + pnl))

    # ------------------------------------------------------
    # Subsection 5.7: Trade Lifecycle Tracking (Exit)
    # ------------------------------------------------------
    # Task: Calculate Cumulative Return & Duration when a tra
de closes.
    if prev_pos != 0 and position == 0:
        # Cumulative Return
        trade_ret = (portfolio[-1] / entry_equity) - 1
        trade_returns.append(trade_ret)
```

```
        # Duration
        duration = t - entry_idx
        trade_durations.append(duration)

        # Store History
        trade_history.append({'Duration': duration, 'Return':
trade_ret})

# ============================================================
==================
# SECTION 6: ADVANCED METRICS CALCULATION
# ============================================================
==================
# Subsection 6.1: Equity & Returns alignment
equity = pd.Series(portfolio[1:], index=data.index[ESTIMATION
_WINDOW:])
returns = equity.pct_change().dropna()

# Subsection 6.2: Standard Risk Metrics (Sharpe, Drawdown, et
c.)
total_ret = (equity.iloc[-1] / equity.iloc[0]) - 1
ann_ret = (1 + total_ret)**(252/len(equity)) - 1
sharpe = np.sqrt(252) * returns.mean() / returns.std()
dd = (equity - equity.cummax()) / equity.cummax()

# Subsection 6.3: Sortino Ratio
downside_returns = returns[returns < 0]
if len(downside_returns) > 0:
    sortino = np.sqrt(252) * returns.mean() / downside_return
s.std()
else:
    sortino = np.inf

# Subsection 6.4: Trade Statistics
trades = np.array(trade_returns)
```

```
if len(trades) > 0:
    win_trades = trades[trades > 0]
    loss_trades = trades[trades < 0]

    win_count = len(win_trades)
    loss_count = len(loss_trades)
    win_rate = (win_count / len(trades)) * 100

    avg_profit = win_trades.mean() if win_count > 0 else 0
    avg_loss = loss_trades.mean() if loss_count > 0 else 0
    max_profit = win_trades.max() if win_count > 0 else 0
    max_loss = loss_trades.min() if loss_count > 0 else 0

    profit_factor = win_trades.sum() / abs(loss_trades.sum())
if loss_count > 0 else np.inf
else:
    win_rate = avg_profit = avg_loss = max_profit = max_loss
= profit_factor = 0
    win_count = loss_count = 0

# Subsection 6.5: Market Beta
common_dates = returns.index.intersection(market_returns.inde
x)
if len(common_dates) > 0:
    cov_matrix = np.cov(returns.loc[common_dates], market_ret
urns.loc[common_dates])
    beta_market = cov_matrix[0][1] / market_returns.loc[commo
n_dates].var()
else:
    beta_market = 0

# Subsection 6.6: Hedging Efficiency
unhedged_returns = data[TICKERS[0]].pct_change().dropna()
common_idx = returns.index.intersection(unhedged_returns.inde
x)
strategy_variance = returns.loc[common_idx].var()
```

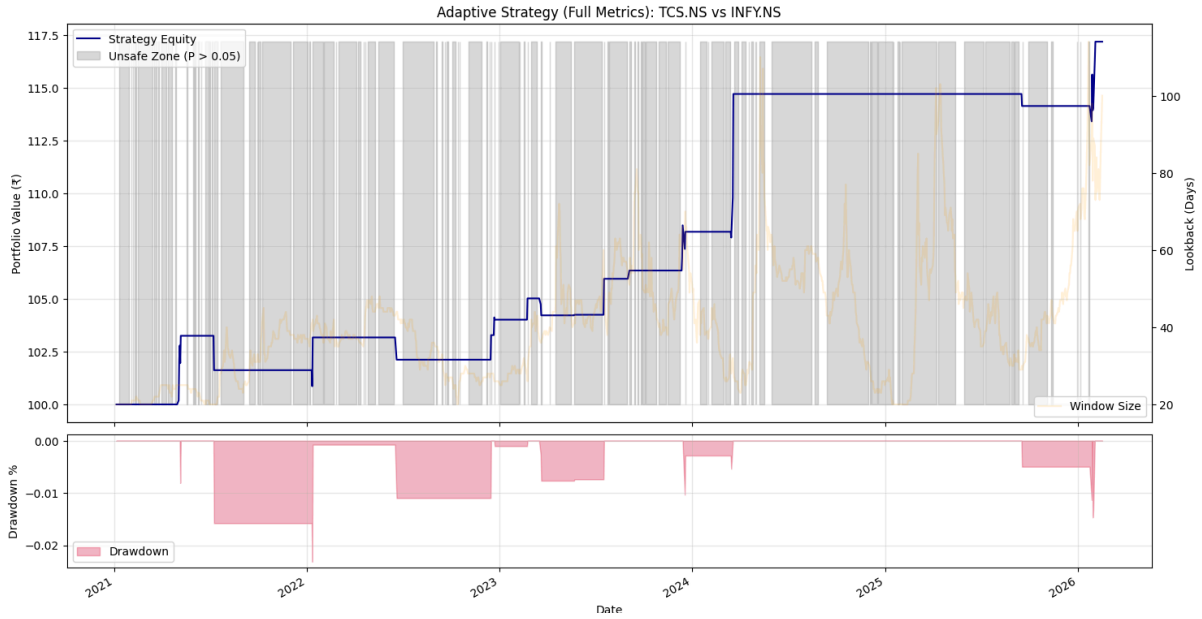
```
unhedged_variance = unhedged_returns.loc[common_idx].var()
hedge_efficiency = 1 - (strategy_variance / unhedged_varianc
e)

# ============================================================
==================
# SECTION 7: VISUALIZATION
# ============================================================
==================
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 10), sharex
=True, gridspec_kw={'height_ratios': [3, 1]})

# Plot 1: Equity Curve
ax1.plot(equity, label="Strategy Equity", color='darkblue', l
inewidth=1.5)
ax1.set_title(f"Adaptive Strategy (Full Metrics): {TICKERS
[0]} vs {TICKERS[1]}")
ax1.set_ylabel("Portfolio Value (₹)")
ax1.grid(True, alpha=0.3)

# Highlight Unsafe Zones
lockout_series = pd.Series(p_value_history, index=equity.inde
x)
unsafe_zones = lockout_series > 0.05
ax1.fill_between(lockout_series.index, equity.min(), equity.m
ax(),
                 where=unsafe_zones, color='grey', alpha=0.3,
label='Unsafe Zone (P > 0.05)')
ax1.legend(loc='upper left')

# Sub-plot: Lookback Window
ax1_sub = ax1.twinx()
ax1_sub.plot(equity.index, dynamic_windows, color='orange', a
lpha=0.15, label='Window Size')
ax1_sub.set_ylabel('Lookback (Days)')
ax1_sub.legend(loc='lower right')
```

```
# Plot 2: Drawdown
ax2.fill_between(dd.index, dd, 0, color='crimson', alpha=0.3,
label="Drawdown")
ax2.set_ylabel("Drawdown %")
ax2.legend()
ax2.grid(True, alpha=0.3)
ax2.set_xlabel("Date")
fig.autofmt_xdate()
plt.tight_layout()

# ============================================================
==================
# SECTION 8: FULL REPORTING
# ============================================================
==================
print(f"{'Metric':<25} | {'Value'}")
print("-" * 40)
print(f"{'Total Return (%)':<25} {total_ret * 100:.2f}%")
print(f"{'Annualized Return':<25} | {ann_ret*100:.2f}%")
print(f"{'Sharpe Ratio':<25} | {sharpe:.2f}")
print(f"{'Sortino Ratio':<25} | {sortino:.2f}")
print(f"{'Max Drawdown':<25} | {dd.min()*100:.2f}%")
print(f"{'Market Beta':<25} | {beta_market:.4f}")
print(f"{'Hedging Efficiency':<25} | {hedge_efficiency*100:.2
f}%")
print("-" * 40)
print(f"{'Total Trades':<25} | {len(trades)}")
print(f"{'Winning Trades':<25} | {win_count}")
print(f"{'Losing Trades':<25} | {loss_count}")
print(f"{'Win Rate':<25} | {win_rate:.2f}%")
print(f"{'Profit Factor':<25} | {profit_factor:.2f}")
print(f"{'Max Profit (1 Trade)':<25} | {max_profit*100:.2
f}%")
print(f"{'Max Loss (1 Trade)':<25} | {max_loss*100:.2f}%")
print(f"{'Avg Profit':<25} | {avg_profit*100:.2f}%")
```

```
print(f"{'Avg Loss':<25} | {avg_loss*100:.2f}%")
print(f"{'% Time in Safety Mode':<25} | {(sum(unsafe_zones)/l
en(unsafe_zones))*100:.1f}%")

# Detailed Trade Log
print("\n" + "="*50)
print(f"{'DETAILED TRADE LOG':^50}")
print("="*50)
print(f"{'Trade #':<10} | {'Duration (Days)':<18} | {'Retur
n':<10}")
print("-" * 50)

if len(trade_history) > 20:
    for i, trade in enumerate(trade_history[:10]):
        print(f"{i+1:<10} | {trade['Duration']:<18} | {trade
['Return']*100:>.2f}%")
    print(f"{'...':<10} | {'...':<18} | {'...':<10}")
    for i, trade in enumerate(trade_history[-10:]):
        idx = len(trade_history) - 10 + i + 1
        print(f"{idx:<10} | {trade['Duration']:<18} | {trade
['Return']*100:>.2f}%")
else:
    for i, trade in enumerate(trade_history):
        print(f"{i+1:<10} | {trade['Duration']:<18} | {trade
['Return']*100:>.2f}%")

print("-" * 50)
if len(trade_durations) > 0:
    print(f"Average Holding Period: {np.mean(trade_duration
s):.1f} days")
else:
    print("No closed trades.")
print("="*50)

plt.show()
```

Outputs of code

The prices of trades are not included in here, sorry for that, but we can have a rough estimate of taxes. As per my knowledge per trades costs around 0.1109%, and I have 15 trades here which comes around 1.6635% which is a huge part of my profit.



Metric	Value
Total Return (%)	17.18%
Annualized Return	3.21%
Sharpe Ratio	0.94
Sortino Ratio	0.44
Max Drawdown	-2.31%
Market Beta	-0.0032
Hedging Efficiency	97.34%
Total Trades	15
Winning Trades	11
Losing Trades	4
Win Rate	73.33%
Profit Factor	5.20
Max Profit (1 Trade)	6.03%
Max Loss (1 Trade)	-1.58%
Avg Profit	1.82%
Avg Loss	-0.96%
% Time in Safety Mode	72.4%

DETAILED TRADE LOG

Trade #	Duration (Days)	Return
1	5	3.25%
2	1	-1.58%
3	2	1.53%
4	1	-1.02%
5	1	1.14%
6	2	0.71%
7	1	0.97%
8	2	-0.76%
9	1	0.03%
10	1	1.63%
11	1	0.37%
12	5	1.73%
13	3	6.03%
14	1	-0.50%
15	6	2.67%

Interpretation of Performance Metrics

The results for the performance metrics consists of : -

- Total returns → 17.18% ; it means in total there is 17.18% of total profit (excluding 1.6635% taxes)
- Annualized returns → 3.21% ; we made 3.21% profits every year on average
- Sharpe ratio → 0.94 ; It should be greater than 1 but in pair trading there are low number of trading signals generated so the profits and losses are inconsistent (most of time zero) decreasing its value but the profits and losses we in narrow range and maximum drawdown was pretty low so it resulted in moderate-bad.
- Sortino ratio → 0.44 ; similarly losses were pretty inconsistent (only 4 signals in 6 years) resulting in such low ratio.
- Maximum Drawdown → -2.31% ; it is pretty low telling us there was not any sudden step peak (profit) followed by deep trough (loss)
- Market Beta → -0.0032 ; it is very near to zero telling us we were not just moving with the flow of market (particularly Nifty 50) but were truly doing statistical arbitrage.
- Hedging Efficiency → 97.34% ; It is also good telling us we were successful in being market neutral.
- Total trades → 15
- Winning trades → 11
- Losing trades → 4
- Win rate → 73.33%
- Profit Factor → 5.20 ; it points to a astonishing profit margins but also tells there are some overfitting going on (which I did 😁).
- Max Profit (1 trade) → 6.03%
- Max Loss (1 trade) → -1.58%
- Average Profit → 1.82%

- Average Loss → -0.96%
- % time in safety mode → 72.4% ; it tells us the main story that most of the times the pair was untradable not totally mean-reverting so we didn't traded in that time.

Also a conclusion I got from considering all the metrics that my strategy is full of many constraints and therefore if market gets volatility I will end up with no trades for a long time. So for this strategy to work well I will need many many pairs so that I can make many trades even if many are not performing.

List of resources used

- Zerodha Varsity Modules
- "Python for Trading: Basic" course of Quantra by QuantInsti
- Use of Gemini in code creation, logic and theory understanding
- Notion for file creation